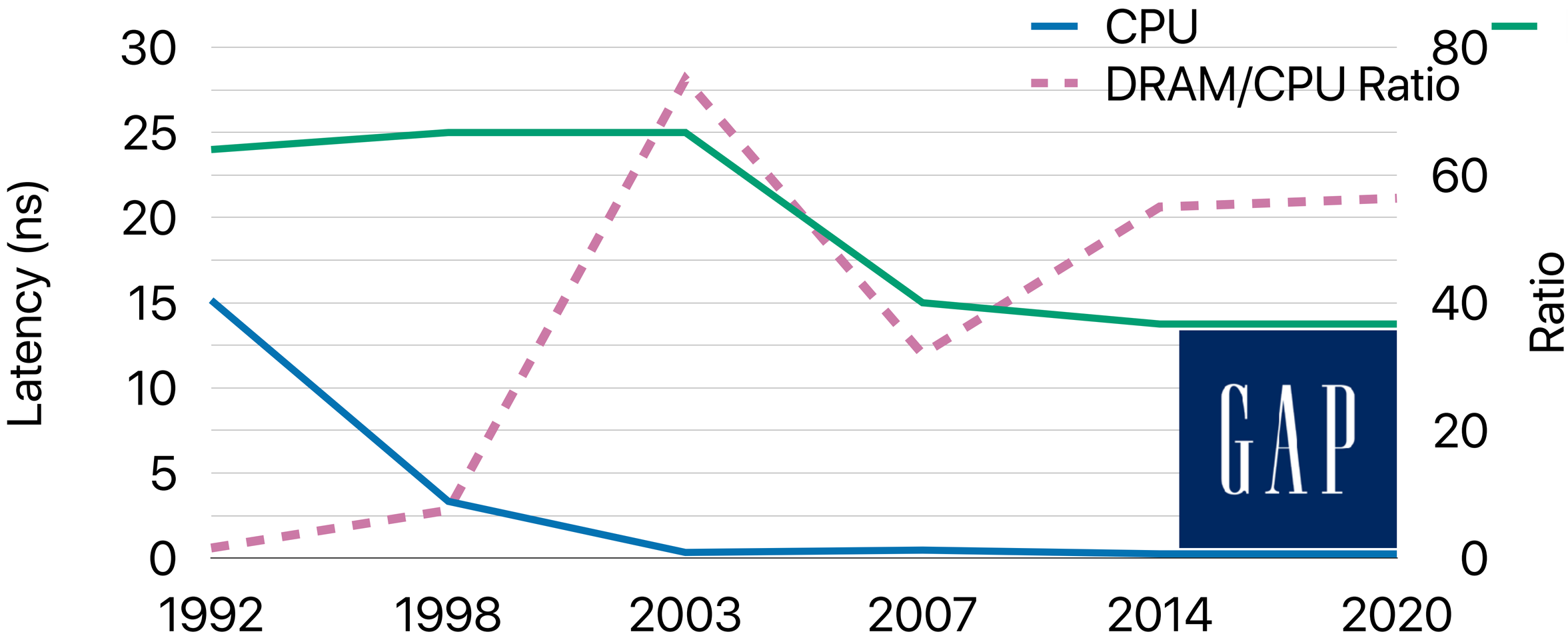
Memory Hierarchy (5): Case study — matrix multiplications

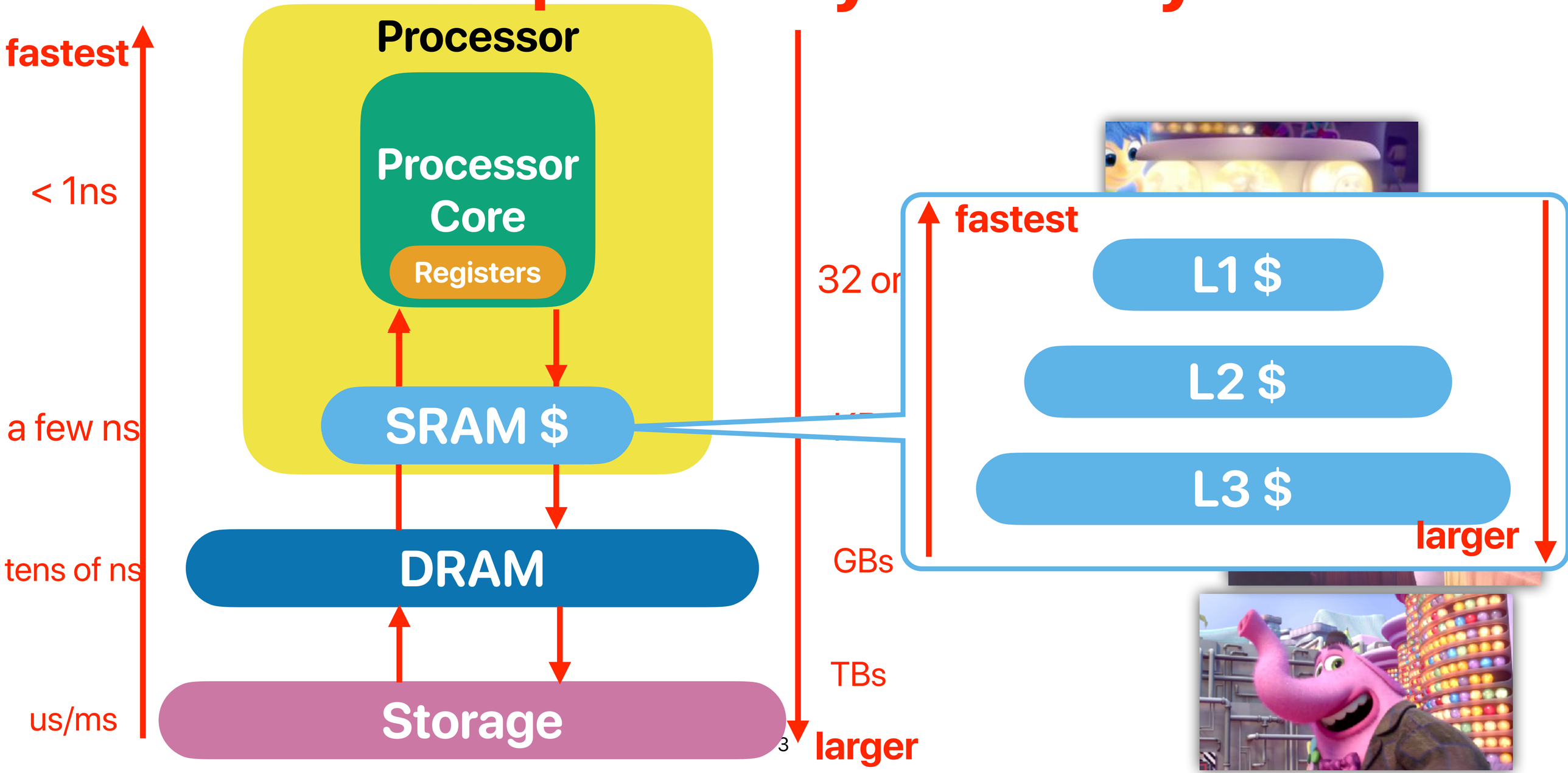
Hung-Wei Tseng

Recap: the “latency” gap between CPU and DRAM



CPU	i486	Pentium II	Pentium 4	Core 2	Core i7-4790K	Core i5-10600K
DRAM	SDRAM	DDR	DDR2	DDR3	DDR4	DDR5

Recap: Memory Hierarchy



Recap: 3Cs of misses

- Compulsory miss
  - Cold start miss. First-time access to a block
- Capacity miss
  - The working set size of an application is bigger than cache size
- Conflict miss
  - Required data replaced by block(s) mapping to the same set
  - Similar collision in hash

Takeaways: Optimizing cache performance through hardware

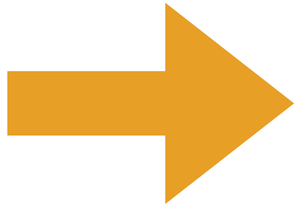
- There is no optimal cache configurations — trade-offs are everywhere
  - Increasing C — (+): capacity misses; (-): cost, access time, power
  - Increasing A — (+): conflict misses; (-): access time, power
  - Increasing B — (+): compulsory misses; (-): miss penalty
- Adding a small buffer alongside the L1 cache can —
  - Virtually add an associative set to frequently used data structures
  - Prefetched blocks won’t cause conflict misses
  - The small buffer can even be tagless to further reduce power consumption and hit time

Loop optimizations

Loop interchange

A

```
for(i = 0; i < ARRAY_SIZE; i++)
{
  for(j = 0; j < ARRAY_SIZE; j++)
  {
    c[i][j] = a[i][j]+b[i][j];
  }
}
```



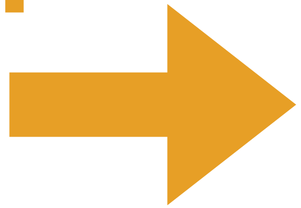
B

```
for(j = 0; j < ARRAY_SIZE; j++)
{
  for(i = 0; i < ARRAY_SIZE; i++)
  {
    c[i][j] = a[i][j]+b[i][j];
  }
}
```

Loop fission

B

```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
}
```



A

```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++)
    e[i] = a[i] * b[i] + c[i];
for(i = 0; i < 512; i++)
    e[i] /= d[i];
```

Loop fusion

A

```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++)
    e[i] = a[i] * b[i] + c[i];
for(i = 0; i < 512; i++)
    e[i] /= d[i];
```



B

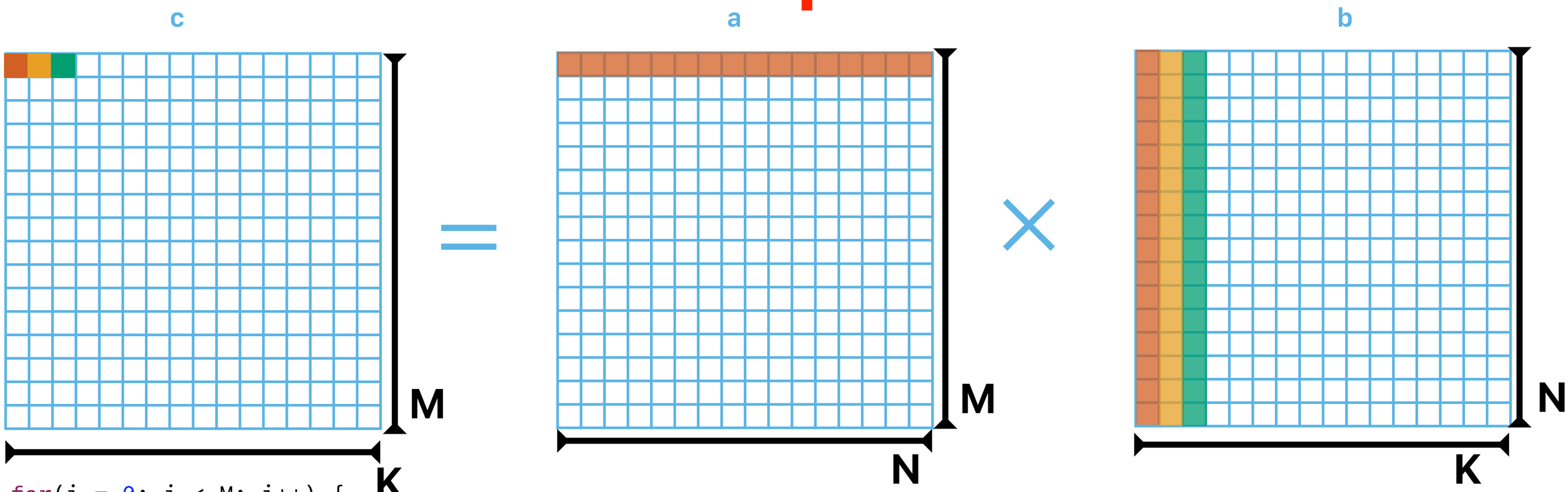
```
double a[8192], b[8192], c[8192], \
       d[8192], e[8192];
for(i = 0; i < 512; i++) {
    e[i] = (a[i] * b[i] + c[i])/d[i];
}
```

Outline

- Case study: matrix multiplications
- Optimizations for matrix multiplications

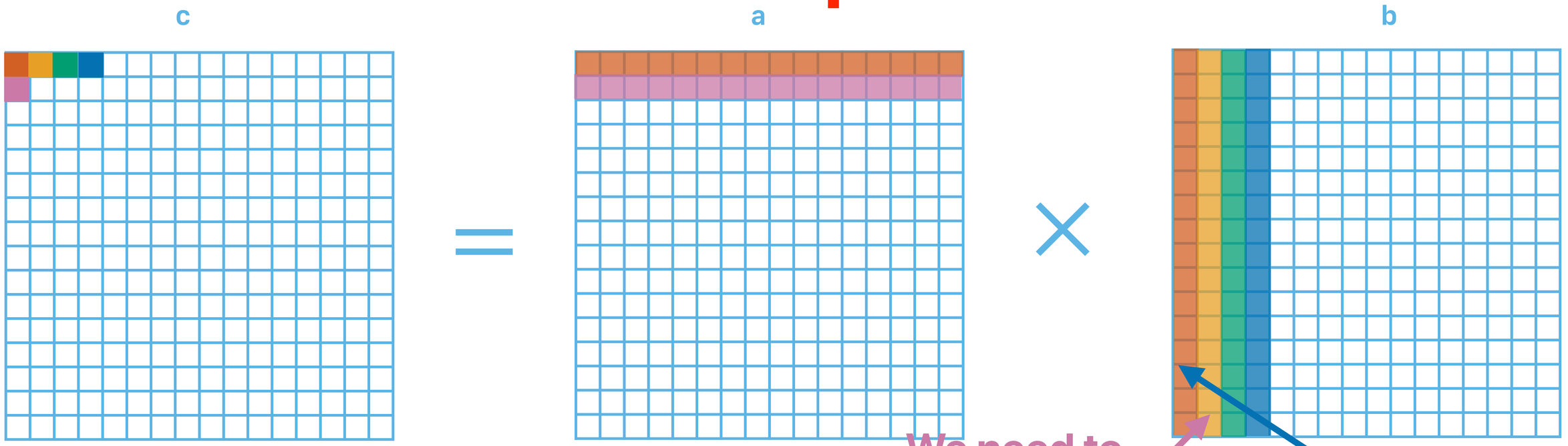
Case study: matrix multiplications

Matrix Multiplications




```
for(i = 0; i < M; i++) {
  for(j = 0; j < K; j++) {
    for(k = 0; k < N; k++) {
      c[i][j] += a[i][k]*b[k][j];
    }
  }
}
```

Matrix Multiplications



- If each dimension of your matrix is 2048
  - Each row takes 2048×8 Bytes = 16 KB
  - Each column takes 2048×8 Bytes = 16 KB
  - The L1-$ of intel Core i7 is 48 KB
  - You can only hold at most 3 rows or 3 columns at most of each matrix!

We need to fetch everything again — capacity miss!

Unlikely to be kept in the cache

What kind(s) of misses are there in Matrix Multiplications

- Considering the case where M=N=K=2048, what do you think the majority type(s) of cache misses are we seeing on an intel processor with intel Core i7 is 48 KB, 12-way, 64-byte blocked L1-$?

```
for(i = 0; i < M; i++) {
  for(j = 0; j < K; j++) {
    for(k = 0; k < N; k++) {
      c[i][j] += a[i][k]*b[k][j];
    }
  }
}
```

A. Compulsory miss
B. Capacity miss
C. Conflict miss
D. Capacity & conflict miss
E. Compulsory & conflict miss

Ideas regarding reducing misses in matrix multiplications

- Reducing capacity misses — we need to reduce the length of a row/column that we visit within a period of time

Mathematical view of MM

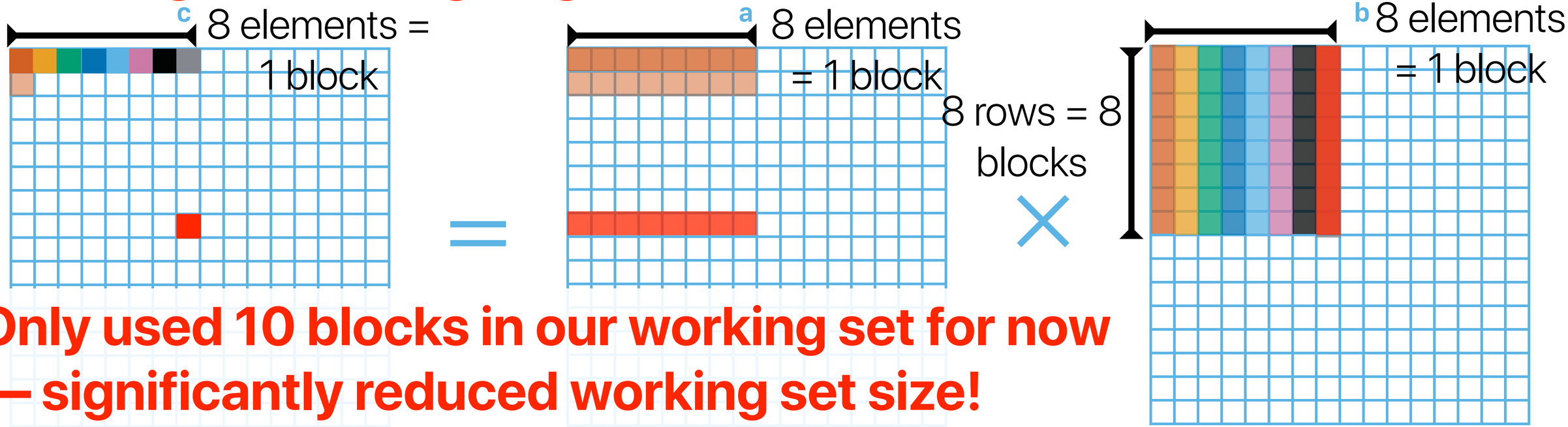
$$c_{i,j} = \sum_{k=0}^{k=N-1} a_{i,k} \times b_{k,j} = \sum_{k=0}^{k=\frac{N}{2}-1} a_{i,k} \times b_{k,j} + \sum_{k=\frac{N}{2}}^{k=N-1} a_{i,k} \times b_{k,j}$$

$$= \sum_{k=0}^{k=\frac{N}{4}-1} a_{i,k} \times b_{k,j} + \sum_{k=\frac{N}{4}}^{k=\frac{N}{2}-1} a_{i,k} \times b_{k,j} + \sum_{k=\frac{N}{2}}^{k=\frac{3N}{4}-1} a_{i,k} \times b_{k,j} + \sum_{k=3N4-1}^{k=N-1} a_{i,k} \times b_{k,j}$$

$$\vdots$$

Let's break up the multiplications and accumulations into something fits in the cache well

Tiling/Blocking Algorithm for Matrix Multiplications



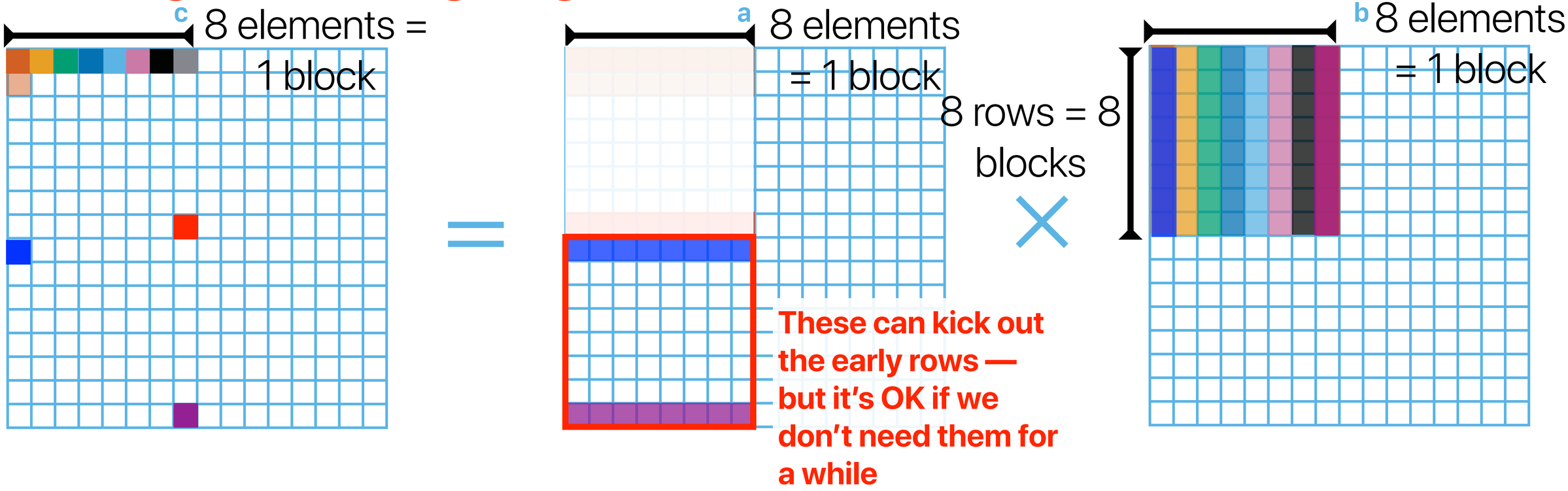
Only used 10 blocks in our working set for now — significantly reduced working set size!

These are still around when we move to the next row in the "tile"

Only compulsory misses —

$$miss_rate = \frac{total\ misses}{total\ accesses} = \frac{8+8+8}{3\times 8\times 8\times 8} = 0.015625$$

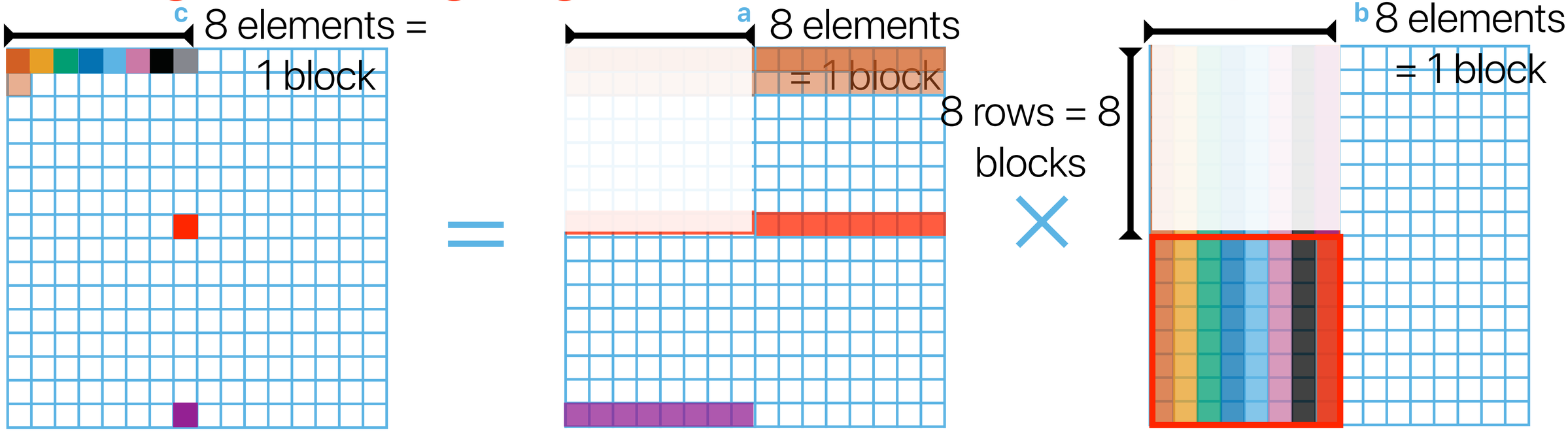
Tiling/Blocking Algorithm for Matrix Multiplications



Bringing miss rate even further lower now —

$$miss_rate = \frac{total\ misses}{total\ accesses} = \frac{8 + 2 \times 8 + 8}{2 \times 3 \times 8 \times 8 \times 8} = 0.0104$$

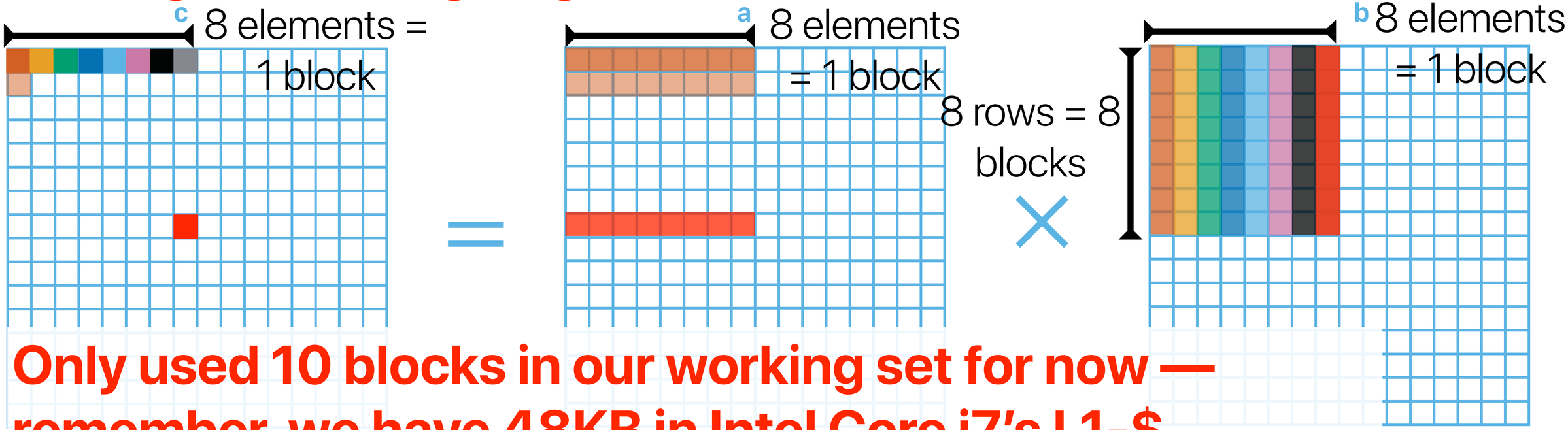
Tiling/Blocking Algorithm for Matrix Multiplications




```
for(i = 0; i < M; i+=tile_size)
    for(j = 0; j < K; j+=tile_size)
        for(k = 0; k < N; k+=tile_size)
            for(ii = i; ii < i+tile_size; ii++)
                for(jj = j; jj < j+tile_size; jj++)
                    for(kk = k; kk < k+tile_size; kk++)
                        c[ii][jj] += a[ii][kk]*b[kk][jj];
```

These can kick out the upper portion of the columns — but it's OK if we don't need them for a while

Tiling/Blocking Algorithm for Matrix Multiplications



Only used 10 blocks in our working set for now — remember, we have 48KB in Intel Core i7's L1-$

We can have at most $\frac{48KB}{3} = 16KB$ for each matrix "tile"

So, let's pick tile size to be 32 as $\sqrt{\frac{48KB}{3 \times 8bytes}} \approx 45$

What if we partition based on cache capacity?

- Considering the case where M=N=K=2048, and a `tile_size=32`, what do you think the majority type(s) of cache misses are we seeing on an intel processor with intel Core i7 is 48 KB, 12-way, 64-byte blocked L1-$?

```
for(i = 0; i < M; i+=tile_size)
   for(j = 0; j < K; j+=tile_size)
       for(k = 0; k < N; k+=tile_size)
          for(ii = i; ii < i+tile_size; ii++)
              for(jj = j; jj < j+tile_size; jj++)
                  for(kk = k; kk < k+tile_size; kk++)
                      c[ii][jj] += a[ii][kk]*b[kk][jj];
```

A. Compulsory miss
B. Capacity miss
C. Conflict miss
D. Capacity & conflict miss
E. Compulsory & conflict miss

Matrix Multiplication — let's consider "b"

```
for(ii = i; ii < i+tile_size; ii++)
    for(jj = j; jj < j+tile_size; jj++)
        for(kk = k; kk < k+tile_size; kk++)
            c[ii][jj] += a[ii][kk]*b[kk][jj];
```

	Address	Tag	Index
b[0][0]	0x20000	0x20	0x0
b[1][0]	0x24000	0x24	0x0
b[2][0]	0x28000	0x28	0x0
b[3][0]	0x2C000	0x2C	0x0
b[4][0]	0x30000	0x30	0x0
b[5][0]	0x34000	0x34	0x0
b[6][0]	0x38000	0x38	0x0
b[7][0]	0x3C000	0x3C	0x0
b[8][0]	0x40000	0x40	0x0
b[9][0]	0x44000	0x44	0x0
b[10][0]	0x48000	0x48	0x0
b[11][0]	0x4C000	0x4C	0x0
b[12][0]	0x50000	0x50	0x0
b[13][0]	0x54000	0x54	0x0
b[14][0]	0x58000	0x58	0x0
b[15][0]	0x5C000	0x5C	0x0
b[16][0]	0x60000	0x60	0x0

- If the row dimension (N) of your matrix is 2048, each row element with the same column index is $2048 \times 8 = 16384 = 0x4000$ away from each other

Each set can store only 12 blocks! So we will start to kick out b[0][0-7], b[1][0-7] …

Now, when we work on c[0][1]

	Address	Tag	Index
b[0][0]	0x20000	0x20	0x0
b[1][0]	0x24000	0x24	0x0
b[2][0]	0x28000	0x28	0x0
b[3][0]	0x2C000	0x2C	0x0
b[4][0]	0x30000	0x30	0x0
b[5][0]	0x34000	0x34	0x0
b[6][0]	0x38000	0x38	0x0
b[7][0]	0x3C000	0x3C	0x0
b[8][0]	0x40000	0x40	0x0
b[9][0]	0x44000	0x44	0x0
b[10][0]	0x48000	0x48	0x0
b[11][0]	0x4C000	0x4C	0x0
b[12][0]	0x50000	0x50	0x0
b[13][0]	0x54000	0x54	0x0
b[14][0]	0x58000	0x58	0x0
b[15][0]	0x5C000	0x5C	0x0
b[16][0]	0x60000	0x60	0x0

	Address	Tag	Index	
b[0][1]	0x20008	0x20	0x0	Conflict
b[1][1]	0x24008	0x24	0x0	Conflict
b[2][1]	0x28008	0x28	0x0	Conflict
b[3][1]	0x2C008	0x2C	0x0	Conflict
b[4][1]	0x30008	0x30	0x0	Conflict
b[5][1]	0x34008	0x34	0x0	Conflict
b[6][1]	0x38008	0x38	0x0	Conflict
b[7][1]	0x3C008	0x3C	0x0	Conflict
b[8][1]	0x40008	0x40	0x0	Conflict
b[9][1]	0x44008	0x44	0x0	Conflict
b[10][1]	0x48008	0x48	0x0	Conflict
b[11][1]	0x4C008	0x4C	0x0	Conflict
b[12][1]	0x50008	0x50	0x0	Conflict
b[13][1]	0x54008	0x54	0x0	Conflict
b[14][1]	0x58008	0x58	0x0	Conflict
b[15][1]	0x5C008	0x5C	0x0	Conflict
b[16][1]	0x60008	0x60	0x0	Conflict

Each set can store only 12 blocks! So we will start to kick out b[0][0-7], b[1][0-7] ...

How large a tile should be?

- Considering the case where M=N=K=2048, and a `tile_size=16`, what do you think the majority type(s) of cache misses are we seeing on an intel processor with intel Core i7 is 48 KB, 12-way, 64-byte blocked L1-$?

```
for(i = 0; i < M; i+=tile_size)
   for(j = 0; j < K; j+=tile_size)
       for(k = 0; k < N; k+=tile_size)
          for(ii = i; ii < i+tile_size; ii++)
              for(jj = j; jj < j+tile_size; jj++)
                  for(kk = k; kk < k+tile_size; kk++)
                      c[ii][jj] += a[ii][kk]*b[kk][jj];
```

A. Compulsory miss
B. Capacity miss
C. Conflict miss
D. Capacity & conflict miss
E. Compulsory & conflict miss

Matrix Multiplication — let's consider "b"

```
for(ii = i; ii < i+tile_size; ii++)
    for(jj = j; jj < j+tile_size; jj++)
        for(kk = k; kk < k+tile_size; kk++)
            c[ii][jj] += a[ii][kk]*b[kk][jj];
```

	Address	Tag	Index
b[0][0]	0x20000	0x20	0x0
b[1][0]	0x24000	0x24	0x0
b[2][0]	0x28000	0x28	0x0
b[3][0]	0x2C000	0x2C	0x0
b[4][0]	0x30000	0x30	0x0
b[5][0]	0x34000	0x34	0x0
b[6][0]	0x38000	0x38	0x0
b[7][0]	0x3C000	0x3C	0x0
b[8][0]	0x40000	0x40	0x0
b[9][0]	0x44000	0x44	0x0
b[10][0]	0x48000	0x48	0x0
b[11][0]	0x4C000	0x4C	0x0
b[12][0]	0x50000	0x50	0x0
b[13][0]	0x54000	0x54	0x0
b[14][0]	0x58000	0x58	0x0
b[15][0]	0x5C000	0x5C	0x0
b[16][0]	0x60000	0x60	0x0

- If the row dimension of your matrix is 2048, each row element with the same column index is $2048 \times 8 = 16384 = 0x4000$ away from each other

If we stop at somewhere before 12 blocks, we should be fine!

Since each block has 8 elements, let's break up in 8 for now

— 8 elements from a[i]

— 8 columns each covers 8 rows

Why is “8” not the best performing?

size	tile_size	IC	Cycles	CPI	CT_ns	ET_s	DL1_miss_rate
2048	4	97765898442	24361786695	0.249185	0.179089	4.362918	0.015038
2048	8	81038403126	21025777083	0.259454	0.179201	3.767848	0.010704
2048	16	74475141550	19295746337	0.259090	0.179058	3.455057	0.071724
2048	32	71542239570	28043594597	0.391987	0.179289	5.027918	0.217212
2048	64	70016931521	32481784024	0.463913	0.179610	5.834055	0.242707
2048	128	69467732816	34433629905	0.495678	0.179135	6.168260	0.246369
2048	256	69144369700	42320146005	0.612055	0.179116	7.580219	0.245532
2048	512	68987473868	47399404451	0.687073	0.179126	8.490458	0.243688

More instructions due to more loop control overhead!

Best performing at 16?

“8” indeed has the best miss rate — and matches our predictions!

Ideas regarding reducing misses in matrix multiplications

- Reducing capacity misses — we need to reduce the length of a row that we visit within a period of time
- Reducing conflict misses — we need to ensure an appropriate tile size would not lead to conflict in sets
- But be careful about the increasing amount of IC/loop control overhead

Use registers wisely

```
for(i = 0; i < M; i+=tile_size)
    for(j = 0; j < K; j+=tile_size)
        for(k = 0; k < N; k+=tile_size)
            for(ii = i; ii < i+tile_size; ii++)
                for(jj = j; jj < j+tile_size; jj++)
                    for(kk = k; kk < k+tile_size; kk++)
                        c[ii][jj] += a[ii][kk]*b[kk][jj];
```

This will create a memory access!

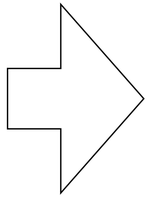
```
for(i = 0; i < M; i+=tile_size)
    for(j = 0; j < K; j+=tile_size)
        for(k = 0; k < N; k+=tile_size)
            for(ii = i; ii < i+tile_size; ii++)
                for(jj = j; jj < j+tile_size; jj++)
                {
                    result = 0;
                    for(kk = k; kk < k+tile_size; kk++)
                        result += a[ii][kk]**b[kk][jj];
                    c[ii][jj] += result;
                }
```

The compiler will try to make result in a register

— without writing code in this way, compiler may not optimize

Matrix Transpose

```
for(i = 0; i < M; i+=tile_size) {
  for(j = 0; j < N; j+=tile_size) {
    for(k = 0; k < K; k+=tile_size) {
      for(ii = i; ii < i+tile_size; ii++)
        for(jj = j; jj < j+tile_size; jj++)
          for(kk = k; kk < k+tile_size; kk++)
            c[ii][jj] += a[ii][kk]*b[kk][jj];
    }
  }
}
```



```
// Transpose matrix b into b_t
for(i = 0; i < ARRAY_SIZE; i+=(ARRAY_SIZE/n)) {
  for(j = 0; j < ARRAY_SIZE; j+=(ARRAY_SIZE/n)) {
      b_t[i][j] += b[j][i];
  }
}
```

```
for(i = 0; i < M; i+=tile_size) {
  for(j = 0; j < N; j+=tile_size) {
    for(k = 0; k < K; k+=tile_size) {
      for(ii = i; ii < i+tile_size; ii++)
        for(jj = j; jj < j+tile_size; jj++)
          result = 0.0;
          for(kk = k; kk < k+tile_size; kk++) {
            // Compute on b_t
            result += a[ii][kk]*b_t[jj][kk];
          }
          c[ii][jj] += result;
    }
  }
}
```

https://www.pollev.com/hungweitseng close in 1:30

What kind(s) of misses can matrix transpose remove?

- By transposing a matrix, the performance of matrix multiplication can be further improved. What kind(s) of cache misses does matrix transpose help to remove?

Block/tile

```
for(i = 0; i < M; i+=tile_size) {
  for(j = 0; j < K; j+=tile_size) {
    for(k = 0; k < N; k+=tile_size) {
      for(ii = i; ii < i+tile_size; ii++)
        for(jj = j; jj < j+tile_size; jj++) {
          result = 0.0;
          for(kk = k; kk < k+tile_size; kk++)
            result += a[ii][kk]*b[kk][jj];
          c[ii][jj] = result;
        }
    }
  }
}
```

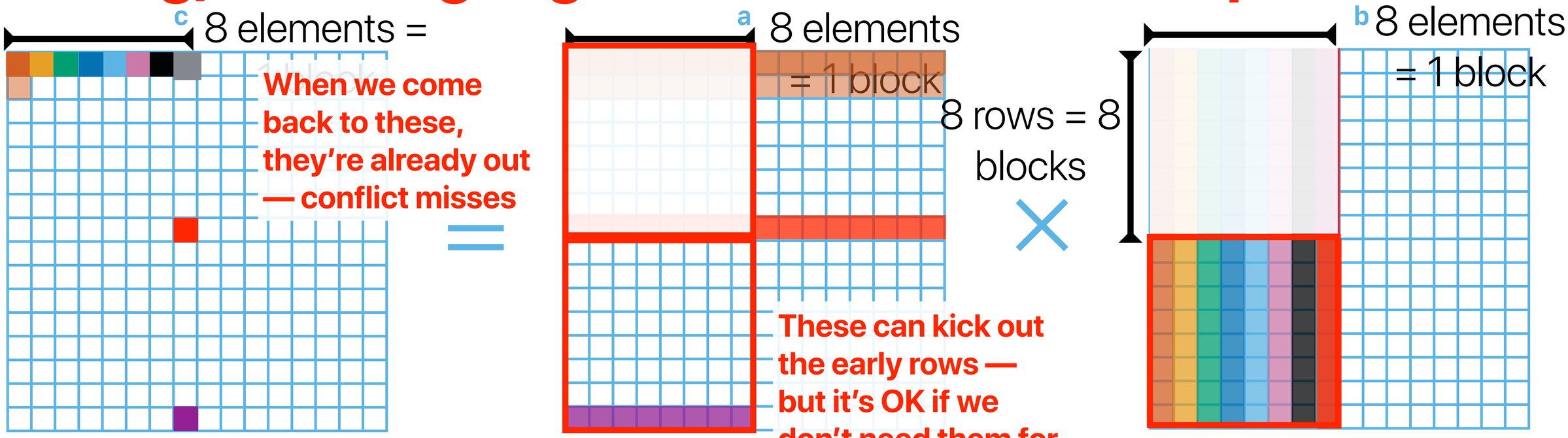
A. Compulsory miss
B. Capacity miss
C. Conflict miss
D. Capacity & conflict miss
E. Compulsory & conflict miss

Block + Transpose

```
// Transpose matrix b into b_t
for(i = 0; i < ARRAY_SIZE; i++) {
  for(j = 0; j < ARRAY_SIZE; j++) {
      b_t[i][j] += b[j][i];
  }
}

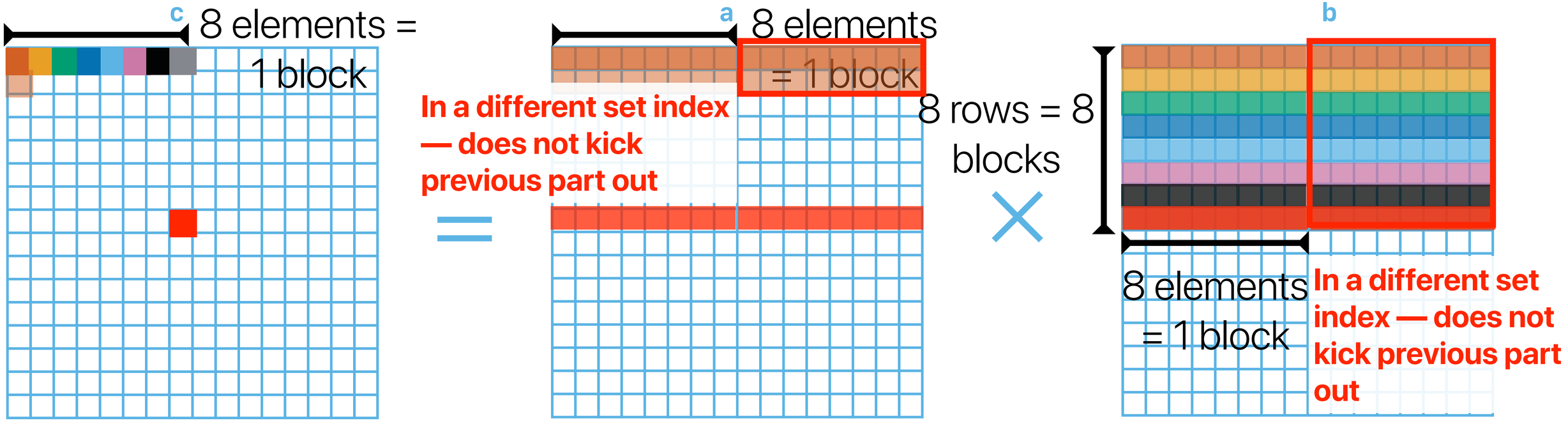

  for(i = 0; i < M; i+=tile_size) {
    for(j = 0; j < K; j+=tile_size) {
      for(k = 0; k < N; k+=tile_size) {
        for(ii = i; ii < i+tile_size; ii++)
          for(jj = j; jj < j+tile_size; jj++)
            result = 0.0;
            for(kk = k; kk < k+tile_size; kk++) {
              // Compute on b_t
              result += a[ii][kk]*b_t[jj][kk];
            }
            c[ii][jj] += result;
    }
  }
}
```

Tiling/Blocking Algorithm for Matrix Multiplications




```
for(i = 0; i < M; i+=tile_size)
    for(j = 0; j < K; j+=tile_size)
        for(k = 0; k < N; k+=tile_size)
            for(ii = i; ii < i+tile_size; ii++)
                for(jj = j; jj < j+tile_size; jj++)
                    for(kk = k; kk < k+tile_size; kk++)
                        c[ii][jj] += a[ii][kk]*b[kk][jj];
```

Tiling/Blocking Algorithm for Transposed Matrix Multiplications



We can make the "tile_size" larger without interfacing conflict misses

```
for(i = 0; i < M; i+=tile_size)
    for(j = 0; j < K; j+=tile_size)
        for(k = 0; k < N; k+=tile_size)
            for(ii = i; ii < i+tile_size; ii++)
                for(jj = j; jj < j+tile_size; jj++)
                    for(kk = k; kk < k+tile_size; kk++)
                        c[ii][jj] += a[ii][kk]*b_t[jj][kk];
```

What kind(s) of misses can matrix transpose remove?

- By transposing a matrix, the performance of matrix multiplication can be further improved. What kind(s) of cache misses does matrix transpose help to remove?

Block/tile

```
for(i = 0; i < M; i+=tile_size) {
  for(j = 0; j < K; j+=tile_size) {
    for(k = 0; k < N; k+=tile_size) {
      for(ii = i; ii < i+tile_size; ii++)
        for(jj = j; jj < j+tile_size; jj++) {
          result = 0.0;
          for(kk = k; kk < k+tile_size; kk++)
            result += a[ii][kk]*b[kk][jj];
          c[ii][jj] = result;
        }
    }
  }
}
```

A. Compulsory miss
B. Capacity miss
C. Conflict miss
D. Capacity & conflict miss
E. Compulsory & conflict miss

Block + Transpose

```
// Transpose matrix b into b_t
for(i = 0; i < ARRAY_SIZE; i++) {
  for(j = 0; j < ARRAY_SIZE; j++) {
      b_t[i][j] += b[j][i];
  }
}


  for(i = 0; i < M; i+=tile_size) {
    for(j = 0; j < K; j+=tile_size) {
      for(k = 0; k < N; k+=tile_size) {
        for(ii = i; ii < i+tile_size; ii++)
          for(jj = j; jj < j+tile_size; jj++)
            result = 0.0;
            for(kk = k; kk < k+tile_size; kk++) {
              // Compute on b_t
              result += a[ii][kk]*b_t[jj][kk];
            }
            c[ii][jj] += result;
    }
  }
}
```

The effect of transposition

Block/tile

size	tile_size	IC	Cycles	CPI	CT_ns	ET_s	DL1_miss_rate
2048	4	106229999405	27619481009	0.259997	0.179102	4.946715	0.017846
2048	8	78811742665	19609444084	0.248814	0.179054	3.511156	0.013409
2048	16	66921533551	15586990700	0.232914	0.179061	2.791023	0.100450
2048	32	61314817568	18373993000	0.299666	0.179289	3.294260	0.296211
2048	64	58579843189	20743873879	0.354113	0.179100	3.715237	0.378661
2048	128	57224253827	20336169181	0.355377	0.179064	3.641477	0.393907
2048	256	56548711458	20103212469	0.355503	0.179245	3.603394	0.398147

Best performing from 16 to 64?

Block + Transpose

size	tile_size	IC	Cycles	CPI	CT_ns	ET_s	DL1_miss_rate
2048	8	70408494537	16474898796	0.233990	0.179050	2.949823	0.003063
2048	16	64863961959	14040845180	0.216466	0.179152	2.515449	0.027740
2048	32	62449092729	15013932980	0.240419	0.179006	2.687589	0.041441
2048	64	[illegible]	[illegible]37	0.224164	0.179182	2.461678	0.023440
2048	128	[illegible]	[illegible]	0.282928	0.178999	3.077470	0.017444
2048	256	60495059818	17659377139	0.291914	0.179154	3.163753	0.024576

Transpose improves the miss rate at larger tile size!

The effect of transposition + register

Block + Transpose

size	tile_size	IC	Cycles	CPI	CT_ns	ET_s	DL1_miss_rate	DL1_accesses
2048	8	70408494537	16474898796	0.233990	0.179050	2.949823	0.003063	28795120136
2048	16	64863961959	14040845180	0.216466	0.179152	2.515449	0.027740	27069370660
2048	32	62449092729	15013932980	0.240419	0.179006	2.687589	0.041441	26388884006
2048	64	61287300182	13738422737	0.224164	0.179182	2.461678	0.023440	26075713629
2048	128	60766823678	17192655415	0.282928	0.178999	3.077470	0.017444	25946188195
2048	256	60495059818	17659377139	0.291914	0.179154	3.163753	0.024576	25877202822

Significant reduction of memory accesses

Block + Transpose + Reg.

size	tile_size	IC	Cycles	CPI	CT_ns	ET_s	DL1_miss_rate	DL1_accesses
2048	8	60790698108	16485243231	0.271180	0.179310	2.955969	0.003943	15892307771
2048	16	49276148749	10288720619	0.208797	0.179004	1.841719	0.066200	12015253818
2048	32	43911066047	9870555579	0.224785	0.179014	1.766965	0.096022	10272381116
2048	64	41301192477	10081924672	0.244107	0.179124	1.805919	0.073499	9437517387
2048	128	40017005995	11757650310	0.293814	0.179214	2.111573	0.049084	9029619561
2048	256	39382447982	14335991956	0.364020	0.179105	2.567647	0.019428	8828955663

Best performing at 32

Tiles do not have to be “squares”

```
for(i = 0; i < M; i+=tile_size_y) {
    for(j = 0; j < K; j+=tile_size_y) {
      for(k = 0; k < N; k+=tile_size_x) {
          for(ii = i; ii < i+tile_size_y; ii++)
              for(jj = j; jj < j+tile_size_y; jj++) {
                  result = 0;
                  for(kk = k; kk < k+tile_size_x; kk++)
                      result += a[ii][kk]*b[jj][kk];
                  c[ii][jj] += result;
              }
          }
      }
   }
}
```

If we could have a rectangular tile + transposition

Block + Transpose + Reg.

size	tile_size	IC	Cycles	CPI	CT_ns	ET_s	DL1_miss_rate
2048	8	60790698108	16485243231	0.271180	0.179310	2.955969	0.003943
2048	16	49276148749	10288720619	0.208797	0.179004	1.841719	0.066200
2048	32	43911066047	9870555579	0.224785	0.179014	1.766965	0.096022
2048	64	41301192477	10081924672	0.244107	0.179124	1.805919	0.073499
2048	128	40017005995	11795578680	0.294764	0.179014	2.111573	0.049084
2048	256	393[illegible]	[illegible]	0.364020	[illegible]	[illegible]	0.019428

Very low miss rate compared to 32x32

Rect. Block + Transpose + Reg.

size	tile_size_x	tile_size_y	IC	Cycles	CPI	CT_ns	ET_s	DL1_miss_rate
2048	8	8	60695609484	11687337167	0.192557	0.179044	2.092552	0.004755
2048	8	16	59756790511	12537990317	0.209817	0.179048	2.244906	0.023099
2048	16	8	49689298094	11447344529	0.230378	0.179035	2.049476	0.004942
2048	16	16	[illegible]	[illegible]	[illegible]	[illegible]9391	1.801964	0.066462
2048	32	16	[illegible]	[illegible]	[illegible]	[illegible]9039	1.685461	0.005974
2048	32	8	43946648680	9760965906	0.222109	0.179051	1.747714	0.064311
2048	64	8	41431489003	9754293903	0.235432	0.179057	1.746575	0.006423
2048	128	8	40059768822	11729101534	0.292790	0.179035	2.099916	0.007373
2048	256	8	39376341663	13824142364	0.351077	0.179305	2.478741	0.003962

Best performing at 32x16

Takeaways: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss
- Loop interchange — conflict/capacity miss
- Loop fission — conflict miss — when $ has limited way associativity
- Loop fusion — capacity miss — when $ has enough way associativity
- Blocking/tiling — capacity miss, conflict miss
- Matrix transpose (a technique changes layout) — conflict misses
- Using registers whenever possible — reduce memory accesses!

Announcement

- Assignment #3 due **next Tuesday**
- Reading quiz #6 due **next Tuesday** before the lecture
- Programming assignment 1 will be up this week
  - Performance based
  - The performance on our server is only for reference, grading based on gradescope server — maybe crowded toward deadline

Computer
Science &
Engineering

203

つづく

